



Node.js va npm

Node.js nima, modullar tizimi (CommonJS va ESM), npm va package.json, fs, path, events, http modullari, muhit o'zgaruvchilari va CLI dasturlar.

O'rta · ~100 daqiqa

Shu paytgacha JavaScript faqat brauzerda ishladi. **Node.js** — JavaScript'ni brauzerdan tashqarida, ya'ni kompyuter yoki serverda ishga tushiruvchi muhit (runtime). U tufayli siz bilgan til bilan web-serverlar, API'lar, CLI dasturlar va avtomatlashtirish skriptlarini yoza olasiz. Bu dars — backend olamiga kirish eshigi.

Node.js nima va qanday ishlaydi?

- 2009-yilda Ryan Dahl yaratgan. Ichida Google Chrome'ning **V8** JavaScript dvigateli ishlaydi.
- **libuv** kutubxonasi orqali fayl tizimi, tarmoq, taymerlar bilan **bloklanmaydigan (non-blocking)** tarzda ishlaydi.
- Brauzerdagi kabi **event loop** modeliga asoslangan (6-dars): bitta oqim minglab ulanishlarga xizmat qiladi, chunki kutish paytida (bazadan javob, fayl o'qish) boshqa so'rovlarni qayta ishlaydi.

Brauzer va Node.js farqi

Brauzer		Node.js
Global obyekt	<code>window</code>	<code>global</code> (umumiy: <code>globalThis</code>)
DOM (<code>document</code>)	Bor	Yo'q
Fayl tizimi	Yo'q (xavfsizlik)	To'liq kirish (<code>fs</code>)

Tarmoq	<code>fetch</code> , WebSocket	<code>fetch</code> , <code>http</code> , TCP/UDP serverlar
Muhit	Har xil brauzer, versiyani tanlay olmaysiz	Versiyani o'zingiz boshqarasiz



Node.js qachon yaxshi tanlov?

I/O ko'p bo'lgan ilovalar: REST API, real vaqtli chat (WebSocket), streaming, mikroservislar. Og'ir CPU hisoblari (video kodlash, katta ML modellari) uchun esa boshqa tillar yoki worker'lar mos.

O'rnatish va birinchi dastur

nodejs.org dan **LTS** versiyani o'rnatish. Bir nechta versiya bilan ishlash uchun `nvm` (Node Version Manager) qulay.

Terminal

```
node --version      # v22.x.x
npm --version       # 10.x.x

node                # REPL – interaktiv rejim (chiqish: .exit yoki Ctrl+C ikki
marta)
> 2 + 2
4

node app.js         # faylni bajarish
node --watch app.js # fayl o'zgarganda avtomatik qayta ishga tushirish (Node
18.11+)
```

app.js

```
console.log('Salom, Node.js!');
console.log('Node versiyasi:', process.version);
console.log('Operatsion tizim:', process.platform);
console.log('Joriy papka:', process.cwd());
console.log('Buyruq argumentlari:', process.argv.slice(2)); // node app.js
ali 25 → ['ali', '25']
```

Modullar tizimi

Node.js'da har bir fayl — alohida modul. Ikki tizim mavjud:

CommonJS (an'anaviy)

math.js

```
function qoshish(a, b) { return a + b; }
const PI = 3.14159;

module.exports = { qoshish, PI };
```

index.js

```
const { qoshish, PI } = require('./math'); // .js kengaytmasi ixtiyoriy
const fs = require('fs');                 // o'rnatilgan modul
const express = require('express');        // npm paketi
console.log(qoshish(2, 3));
```

ES Modules (zamonaviy, tavsiya etiladi)

```
package.json

{
  "type": "module"
}
```

```
math.js

export const qoshish = (a, b) => a + b;
export default class Kalkulyator { /* ... */ }
```

```
index.js

import Kalkulyator, { qoshish } from './math.js'; // ESM'da kengaytma
MAJBURIY
import fs from 'node:fs/promises'; // "node:" prefiksi –
o'rnatilgan modul ekanini aniq ko'rsatadi
import express from 'express';

const config = JSON.parse(await fs.readFile('./config.json', 'utf8')); // top-
level await
```

	CommonJS	ES Modules
Import	<code>require()</code>	<code>import</code>
Eksport	<code>module.exports</code>	<code>export</code>
Yuklanishi	Sinxron	Asinxron
Top-level await	Yo'q	Bor
<code>__dirname</code>	Bor	<code>import.meta.dirname</code> (Node 20.11+)

Ushbu qo'llanmaning qolgan qismida **ES Modules** ishlatamiz.

npm va package.json

npm (Node Package Manager) — dunyodagi eng katta dasturiy kutubxonalar ombori (2 milliondan ortiq paket) va ularni boshqaruvchi vosita.

Terminal

```
npm init -y                # package.json yaratish

npm install express        # dependency (ilova ishlashi uchun kerak)
npm i express              # qisqa yozuv
npm install -D nodemon eslint # devDependency (faqat ishlab chiqishda)
npm install -g npm-check-updates # global (buyruq sifatida)

npm install                # package.json dagi barcha paketlarni o'rnatish
(clone'dan keyin)

npm ci                    # lock-fayl bo'yicha aniq o'rnatish (CI/server
uchun)

npm uninstall lodash

npm outdated              # eskirgan paketlar
npm audit                 # xavfsizlik zaifliklari
npx create-vite@latest    # paketni o'rnatmasdan bir martalik bajarish
```

package.json

```
{
  "name": "kitob-api",
  "version": "1.0.0",
  "description": "Kitoblar do'koni uchun REST API",
  "type": "module",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "node --test",
    "lint": "eslint src"
  },
  "dependencies": {
    "express": "^5.1.0",
    "pg": "^8.13.0"
  },
  "devDependencies": {
    "eslint": "^9.0.0"
  },
  "engines": {
    "node": "≥ 20"
  }
}
```

- **scripts** — `npm run dev`, `npm test`, `npm start` orqali ishga tushiriladigan buyruqlar.
- **Semantik versiyalash (SemVer)** — `MAJOR.MINOR.PATCH`: `5.1.0`. MAJOR — mos kelmaydigan o'zgarishlar, MINOR — yangi funksiyalar, PATCH — xato tuzatishlar.
- `^5.1.0` — 5.x.x ichida yangilanishga ruxsat; `~5.1.0` — faqat 5.1.x; `5.1.0` — aynan shu versiya.
- **package-lock.json** — har bir paketning aniq o'rnatilgan versiyasini qayd qiladi. **Git'ga commit qiling** — jamoadagi hamma bir xil versiyalarni oladi.
- **node_modules/** — o'rnatilgan paketlar. **Hech qachon commit qilmang** (`.gitignore`).



Paket tanlashda ehtiyot bo'ling

Har bir paket — boshqa bironing kodi serveringizda ishlashi demak. Haftalik yuklanishlar soni, oxirgi yangilanish sanasi, GitHub'dagi faollik va `npm audit` natijasiga qarang. Nomi o'xshash soxta paketlardan (typosquatting: `expres`, `loadsh`) saqlaning.

O'rnatilgan asosiy modullar

fs — fayl tizimi

JavaScript

```
import fs from 'node:fs/promises';
import { existsSync, createReadStream } from 'node:fs';

// O'qish va yozish
const matn = await fs.readFile('eslatma.txt', 'utf8');
await fs.writeFile('chiqish.txt', 'Salom\n'); // yaratadi yoki
ustidan yozadi
await fs.appendFile('log.txt', `${new Date().toISOString()} - kirish\n`);

// JSON fayl
const data = JSON.parse(await fs.readFile('data.json', 'utf8'));
await fs.writeFile('data.json', JSON.stringify(data, null, 2));

// Papkalar
await fs.mkdir('uploads/2026', { recursive: true });
const fayllar = await fs.readdir('uploads');
const info = await fs.stat('data.json'); // info.size, info.mtime,
info.isDirectory()
await fs.rename('eski.txt', 'yangi.txt');
await fs.rm('temp', { recursive: true, force: true });

if (!existsSync('config.json')) console.log('Config topilmadi');
```



`fs.readFileSync` kabi sinxron funksiyalar event loop'ni bloklaydi. Ularni faqat dastur ishga tushayotganda (konfiguratsiya o'qish) yoki CLI skriptlarda ishlating; server so'rovlarini qayta ishlashda — doim asinxron versiyalar.

path — fayl yo'llari

JavaScript

```
import path from 'node:path';

path.join('uploads', '2026', 'rasm.png'); // 'uploads/2026/rasm.png'
// Windows'da '\\'
path.resolve('src', 'app.js'); // to'liq absolyut yo'l
path.extname('rasm.final.png'); // '.png'
path.basename('/a/b/rasm.png'); // 'rasm.png'
path.basename('/a/b/rasm.png', '.png'); // 'rasm'
path.dirname('/a/b/rasm.png'); // '/a/b'

// Joriy fayl papkasi (ESM)
const __dirname = import.meta.dirname;
const publicDir = path.join(__dirname, 'public');
```

Yo'llarni hech qachon satr qo'shish bilan (`'uploads/' + name`) yasamang — `path.join` Windows/Linux farqini va ortiqcha `/` larni o'zi hal qiladi.

events — hodisalar

JavaScript

```
import { EventEmitter } from 'node:events';

class Buyurtmalar extends EventEmitter {
  yarat(buyurtma) {
    // ... bazaga saqlash ...
    this.emit('yaratildi', buyurtma);
  }
}

const orders = new Buyurtmalar();
orders.on('yaratildi', (b) => console.log(`✉️ ${b.email} ga tasdiq xati yuborildi`));
orders.on('yaratildi', (b) => console.log(`📦 Omborga xabar: ${b.mahsulot}`));
orders.once('yaratildi', () => console.log(`🎉 Birinchi buyurtma!`));

orders.yarat({ email: 'ali@mail.uz', mahsulot: 'Noutbuk' });
```

Node.js'ning ko'plab qismlari (server, oqimlar) `EventEmitter` ustiga qurilgan. Bu usul kodni bir-biridan mustaqil qismlarga ajratadi.

Streams — oqimlar

1 GB faylni `readFile` bilan o'qish 1 GB xotira talab qiladi. **Stream** esa ma'lumotni kichik bo'laklarda (chunk) qayta ishlaydi:

JavaScript

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';

// Katta log faylni siqish — xotira sarfi bir necha KB
await pipeline(
  createReadStream('access.log'),
  createGzip(),
  createWriteStream('access.log.gz'),
);
```

process va muhit o'zgaruvchilari

Parollar, API kalitlari, port raqami kabi sozlamalar kodda emas, **muhit o'zgaruvchilarida (environment variables)** saqlanadi.

.env

```
PORT=3000
DATABASE_URL=postgres://user:parol@localhost:5432/kitoblar
JWT_SECRET=juda-uzun-va-tasodifiy-maxfiy-kalit
NODE_ENV=development
```

JavaScript

```
// Node 20.6+ da .env faylni o'rnatilgan holda yuklash:
//   node --env-file=.env src/server.js
// (eski versiyalarda: npm i dotenv → import 'dotenv/config';)

const PORT = Number(process.env.PORT) || 3000;
const isProd = process.env.NODE_ENV === 'production';

if (!process.env.JWT_SECRET) {
  console.error('✗ JWT_SECRET o\'rnatilmagan');
  process.exit(1); // xato kodi bilan chiqish
}

process.on('SIGINT', () => { // Ctrl+C
  console.log('Server to\'xtatilmoqda...');
  process.exit(0);
});
```

Repozitoriyaga `.env` emas, uning namunasini — `.env.example` ni (qiymatlarsiz) commit qiling.

http moduli — freymvorksiz server

Express'ga o'tishdan oldin, uning ostida nima borligini ko'raylik:

```

import http from 'node:http';

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  // So'rov tanasini o'qish — stream bo'laklarini yig'amiz
  let body = '';
  for await (const chunk of req) body += chunk;

  console.log(req.method, url.pathname, url.searchParams.toString(), body);

  res.setHeader('Content-Type', 'application/json; charset=utf-8');

  if (req.method === 'GET' && url.pathname === '/api/salom') {
    const ism = url.searchParams.get('ism') ?? 'Mehmon';
    res.end(JSON.stringify({ xabar: `Salom, ${ism}!` }));
  } else if (req.method === 'POST' && url.pathname === '/api/echo') {
    try {
      const data = JSON.parse(body);
      res.statusCode = 201;
      res.end(JSON.stringify({ olindi: data }));
    } catch {
      res.statusCode = 400;
      res.end(JSON.stringify({ xato: 'JSON noto\'g\'ri' }));
    }
  } else {
    res.statusCode = 404;
    res.end(JSON.stringify({ xato: 'Topilmadi' }));
  }
});

server.listen(3000, () => console.log('http://localhost:3000'));

```

Ko'rib turganingizdek: marshrutlash (`if/else`), tanani o'qish, JSON'ni parse qilish, xatolar — hammasi qo'lda. Marshrutlar 50 taga yetsa, bu kod boshqarib bo'lmaydigan bo'ladi. Aynan shu muammoni **Express** hal qiladi (9-dars).

Amaliy misollar

MISOL 1

Yuklamalar papkasini tartiblovchi CLI dastur

"Downloads" papkasidagi yuzlab fayllarni turiga qarab papkalarga ajratadigan foydali skript.

```

#!/usr/bin/env node
import fs from 'node:fs/promises';
import path from 'node:path';

const TOIFALAR = {
  Rasmlar: ['.jpg', '.jpeg', '.png', '.gif', '.webp', '.svg'],
  Hujjatlar: ['.pdf', '.doc', '.docx', '.xls', '.xlsx', '.ppt', '.pptx', '.txt'],
  Videolar: ['.mp4', '.mkv', '.avi', '.mov'],
  Musiqa: ['.mp3', '.wav', '.flac'],
  Arxivlar: ['.zip', '.rar', '.7z', '.tar', '.gz'],
  Dasturlar: ['.exe', '.msi', '.dmg', '.deb'],
};

// Kengaytma → toifa (tez qidirish uchun teskari jadval)
const kengaytmaToifa = Object.fromEntries(
  Object.entries(TOIFALAR).flatMap(([toifa, exts]) ⇒ exts.map(e ⇒ [e, toifa]))
);

const [, , papka = '.', ...flags] = process.argv;
const sinov = flags.includes('--dry-run'); // faqat ko'rsatish, ko'chirmaslik

async function tartibla(dir) {
  const elementlar = await fs.readdir(dir, { withFileTypes: true });
  const hisobot = {};

  for (const el of elementlar) {
    if (!el.isFile() || el.name.startsWith('.')) continue;

    const toifa = kengaytmaToifa[path.extname(el.name).toLowerCase()] ?? 'Boshqalar';
    const manzilPapka = path.join(dir, toifa);
    let manzil = path.join(manzilPapka, el.name);

    // Bir xil nomli fayl bo'lsa: rasm.png → rasm (1).png
    for (let i = 1; await mavjudmi(manzil); i++) {
      const { name, ext } = path.parse(el.name);
      manzil = path.join(manzilPapka, `${name} (${i})${ext}`);
    }
  }
}

```

```

    if (!sinov) {
      await fs.mkdir(manzilPapka, { recursive: true });
      await fs.rename(path.join(dir, el.name), manzil);
    }
    hisobot[toifa] = (hisobot[toifa] ?? 0) + 1;
    console.log(`${sinov ? '[sinov] ' : ''}${el.name} → ${toifa}/`);
  }
  return hisobot;
}

async function mavjudmi(p) {
  try { await fs.access(p); return true; } catch { return false; }
}

try {
  const natija = await tartibla(path.resolve(papka));
  console.log(`\n📊 Natija:`);
  console.table(natija);
} catch (err) {
  console.error(`❌ Xato: ${err.code === 'ENOENT' ? 'papka topilmadi' : err.message}`);
  process.exit(1);
}

```

Terminal

```

node organize.js ~/Downloads --dry-run # avval nima bo'lishini ko'ramiz
node organize.js ~/Downloads          # haqiqiy ko'chirish

```

`--dry-run` rejimi — xavfli amallarni bajaruvchi har qanday skriptda bo'lishi kerak bo'lgan yaxshi odat.

MISOL 2 Statik fayl serveri (http + fs + path)

Shaxsiy saytingizni (2–3-darslar) brauzerda `http://localhost:8080` orqali ochadigan server. Unda real serverlar duch keladigan muammolar — MIME turlari, 404 va xavfsizlik — hal qilingan.

```

import http from 'node:http';
import fs from 'node:fs/promises';
import { createReadStream } from 'node:fs';
import path from 'node:path';

const ROOT = path.resolve(process.argv[2] ?? 'public');
const PORT = Number(process.env.PORT) || 8080;

const MIME = {
  '.html': 'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js': 'text/javascript; charset=utf-8',
  '.json': 'application/json',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon',
  '.woff2': 'font/woff2',
};

const server = http.createServer(async (req, res) => {
  const started = Date.now();
  res.on('finish', () =>
    console.log(`${req.method} ${req.url} → ${res.statusCode} (${Date.now() - started}ms)`
  );
  try {
    const { pathname } = new URL(req.url, 'http://localhost');
    let filePath = path.join(ROOT, decodeURIComponent(pathname));

    // 🚫 Path traversal hujumidan himoya: ../../etc/passwd
    if (filePath !== ROOT && !filePath.startsWith(ROOT + path.sep)) {
      res.writeHead(403).end('Forbidden');
      return;
    }

    let stat = await fs.stat(filePath).catch(() => null);
    if (stat?.isDirectory()) {
      filePath = path.join(filePath, 'index.html');
      stat = await fs.stat(filePath).catch(() => null);
    }
  }

```

```

    if (!stat) {
      res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8' });
      res.end('<h1>404</h1><p>Sahifa topilmadi</p>');
      return;
    }

    res.writeHead(200, {
      'Content-Type': MIME[path.extname(filePath)] ?? 'application/octet-
stream',
      'Content-Length': stat.size,
      'Cache-Control': 'public, max-age=3600',
    });
    createReadStream(filePath).pipe(res); // stream – katta fayllar ham
xotirani to'ldirmaydi
  } catch (err) {
    console.error(err);
    res.writeHead(500).end('Server xatosi');
  }
});

server.listen(PORT, () => console.log(`📁 ${ROOT} →
http://localhost:${PORT}`));

```

Terminal

```

node static-server.js ./portfolio
curl -i http://localhost:8080/
curl -i --path-as-is http://localhost:8080/../../etc/passwd # 403
qaytishi kerak

```

MISOL 3 JSON faylga asoslangan "mini ma'lumotlar bazasi" moduli

Haqiqiy bazaga (10-dars) o'tishdan oldin, ma'lumotni JSON faylda saqlovchi qayta ishlatiladigan modul yozamiz. U 9-darsdagi Express API'da ishlatiladi.

```

import fs from 'node:fs/promises';
import path from 'node:path';
import { randomUUID } from 'node:crypto';

export class JsonStore {
  #file;
  #queue = Promise.resolve(); // yozuvlarni navbatga qo'yish uchun

  constructor(file) {
    this.#file = path.resolve(file);
  }

  async #read() {
    try {
      return JSON.parse(await fs.readFile(this.#file, 'utf8'));
    } catch (err) {
      if (err.code === 'ENOENT') return []; // fayl hali yo'q
      throw err;
    }
  }

  // Bir vaqtdagi ikki yozuv bir-birini o'chirib yubormasligi uchun -
  // ketma-ket bajaramiz
  #write(mutator) {
    this.#queue = this.#queue.then(async () => {
      const items = await this.#read();
      const result = mutator(items);
      const tmp = `${this.#file}.tmp`;
      await fs.mkdir(path.dirname(this.#file), { recursive: true });
      await fs.writeFile(tmp, JSON.stringify(items, null, 2));
      await fs.rename(tmp, this.#file); // atomar almashtirish -
      // fayl hech qachon yarim yozilmaydi
      return result;
    });
    return this.#queue;
  }

  async all({ filter = () => true, sort } = {}) {
    const items = (await this.#read()).filter(filter);
    return sort ? items.sort(sort) : items;
  }
}

```

```

async get(id) {
  return (await this.#read()).find(i => i.id === id) ?? null;
}

create(data) {
  return this.#write(items => {
    const item = { id: randomUUID(), ...data, createdAt: new
Date().toISOString() };
    items.push(item);
    return item;
  });
}

update(id, patch) {
  return this.#write(items => {
    const i = items.findIndex(x => x.id === id);
    if (i === -1) return null;
    items[i] = { ...items[i], ...patch, id, updatedAt: new
Date().toISOString() };
    return items[i];
  });
}

remove(id) {
  return this.#write(items => {
    const i = items.findIndex(x => x.id === id);
    if (i === -1) return false;
    items.splice(i, 1);
    return true;
  });
}
}

```

try.js

```
import { JsonStore } from './src/db/jsonStore.js';

const books = new JsonStore('data/books.json');
const b = await books.create({ title: 'Clean Code', author: 'Robert Martin', price: 180000 });
await books.update(b.id, { price: 165000 });
console.log(await books.all({ sort: (a, b) => a.price - b.price }));

// Bir vaqtda 10 ta yozuv – navbat tufayli birortasi ham yo'qolmaydi
await Promise.all(Array.from({ length: 10 }, (_, i) => books.create({ title: `Kitob ${i}` })));
console.log((await books.all()).length); // 11
```



Navbatsiz versiyada parallel `create` chaqiruvlari bir xil eski faylni o'qib, bir-birining natijasini ustidan yozib yuborardi — klassik **race condition**. Haqiqiy ma'lumotlar bazalari bu muammoni tranzaksiyalar bilan hal qiladi (10-dars).

MISOL 4

Valyuta kursi CLI: tashqi API, argumentlar va kesh

O'zbekiston Markaziy banki ochiq API'sidan kurslarni olib, summani konvertatsiya qiladi. Natija 1 soat keshlanadi.

```

#!/usr/bin/env node
import fs from 'node:fs/promises';
import os from 'node:os';
import path from 'node:path';
import { parseArgs } from 'node:util';

const { values, positionals } = parseArgs({
  allowPositionals: true,
  options: {
    to: { type: 'string', short: 't', default: 'UZS' },
    fresh: { type: 'boolean', default: false },
    help: { type: 'boolean', short: 'h' },
  },
});

if (values.help || positionals.length < 2) {
  console.log(`Foydalanish: kurs <summa> <valyuta> [--to UZS] [--fresh]
Misollar:
  node kurs.js 100 USD
  node kurs.js 1000000 UZS --to EUR`);
  process.exit(0);
}

const [summaStr, from] = positionals;
const summa = Number(summaStr);
const CACHE = path.join(os.tmpdir(), 'cbu-rates.json');
const TTL = 60 * 60 * 1000;

async function kurslarniOl() {
  if (!values.fresh) {
    try {
      const { savedAt, rates } = JSON.parse(await fs.readFile(CACHE,
'utf8'));
      if (Date.now() - savedAt < TTL) return rates;
    } catch { /* kesh yo'q — davom etamiz */ }
  }
  const res = await fetch('https://cbu.uz/uz/arkhiv-kursov-valyut/json/',
{
    signal: AbortSignal.timeout(10_000),
  });
  if (!res.ok) throw new Error(`CBU API: ${res.status}`);
  const list = await res.json();

```

```

// [{ Ccy: 'USD', Rate: '12650.15', Nominal: '1', Date: '03.10.2026',
... }]

const rates = { UZS: 1 };
for (const r of list) rates[r.Ccy] = Number(r.Rate) / Number(r.Nominal);
await fs.writeFile(CACHE, JSON.stringify({ savedAt: Date.now(), rates
}));
return rates;
}

try {
  if (!Number.isFinite(summa) || summa < 0) throw new Error('Summa
noto\'g\'ri');
  const rates = await kurslarniOl();
  const a = from.toUpperCase(), b = values.to.toUpperCase();
  if (!rates[a] || !rates[b]) throw new Error(`Noma'lum valyuta:
${!rates[a] ? a : b}`);

  const natija = (summa * rates[a]) / rates[b];
  const f = (n, c) => new Intl.NumberFormat('uz-UZ', {
maximumFractionDigits: 2 }).format(n) + ' ' + c;
  console.log(`💰 ${f(summa, a)} = ${f(natija, b)}`);
  console.log(`1 ${a} = ${f(rates[a] / rates[b], b)}`);
} catch (err) {
  console.error('❌', err.message);
  process.exitCode = 1;
}

```

Terminal

```

node kurs.js 100 USD
node kurs.js 5000000 UZS --to EUR
node kurs.js 50 EUR -t USD --fresh

```

`package.json` ga `"bin": { "kurs": "./kurs.js" }` qo'shib, `npm link` qilsangiz, buyruq butun tizimda `kurs 100 USD` ko'rinishida ishlaydi.

Mashqlar



Mustaqil ish

1. `wc.js` yozing: berilgan fayldagi qatorlar, so'zlar va belgilar sonini chiqarsin (`node wc.js matn.txt`).
2. Papkadagi barcha `.jpg` fayllar nomini `2026-10-03_001.jpg` formatiga o'zgartiruvchi skript (fayl yaratilgan sana bo'yicha).
3. 8-darsdagi `http` serveriga `GET /api/notes` va `POST /api/notes` marshrutlarini qo'shing, ma'lumot 3-misoldagi `JsonStore` da saqlansin.
4. `EventEmitter` asosida oddiy "Pomodoro taymer" yozing: `start`, `tick`, `break`, `done` hodisalari bilan.
5. Loyihangizga `.env` va `.env.example` qo'shing; PORT o'rnatilmagan bo'lsa ogohlantirish chiqaring.



Xulosa

- Node.js = V8 + libuv: JavaScript serverda, bloklanmaydigan I/O bilan.
- ES Modules (`"type": "module"`, `import/export`) — zamonaviy standart.
- `package.json` — loyiha pasporti; `package-lock.json` ni commit qiling, `node_modules` ni — yo'q.
- `fs/promises`, `path`, `events`, `streams` — eng ko'p ishlatiladigan modullar.
- Maxfiy sozlamalar — `.env` va `process.env` da.
- Sof `http` bilan server yozish mumkin, lekin katta ilovalar uchun freymvork kerak — keyingi dars Express.

☐ Darsni o'qib bo'ldim